VIDEO NOTES · PART 2

What Is LangChain, and Why Use It?

The framework's core components, explained simply

What this document is

A plain-English walkthrough of the seven building blocks of LangChain — the problem each one solves, and how they fit together.

1 How AI Development Changed

To understand why LangChain exists, it helps to see what came before it.

In **traditional Machine Learning**, most of the effort went into the model itself. You collected data, trained a model for one specific task, and the model was the hard part of the project.

With **Generative AI**, that flipped. The model already exists and is extremely capable — you simply call it. The hard part moved to everything *around* the model.

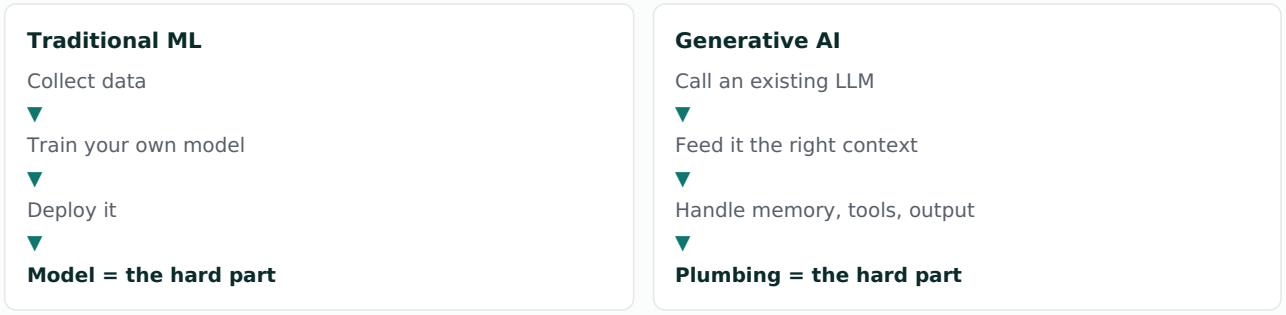

Figure 1 — The difficulty moved from building the model to wiring it into an application.

2 The Glue Code Problem

Before frameworks like LangChain, every developer building an LLM application had to write the same supporting code by hand, over and over. This is often called **glue code** — the code that exists only to hold other pieces together.

Every project needed its own version of prompt handling, its own way of storing chat history, its own logic for pulling in documents, and its own parsing of whatever text the model returned.

> **The result:** messy, repetitive codebases that were painful to debug. When something went wrong, it was rarely clear whether the fault was in the prompt, the retrieval step, the parsing, or the model itself.

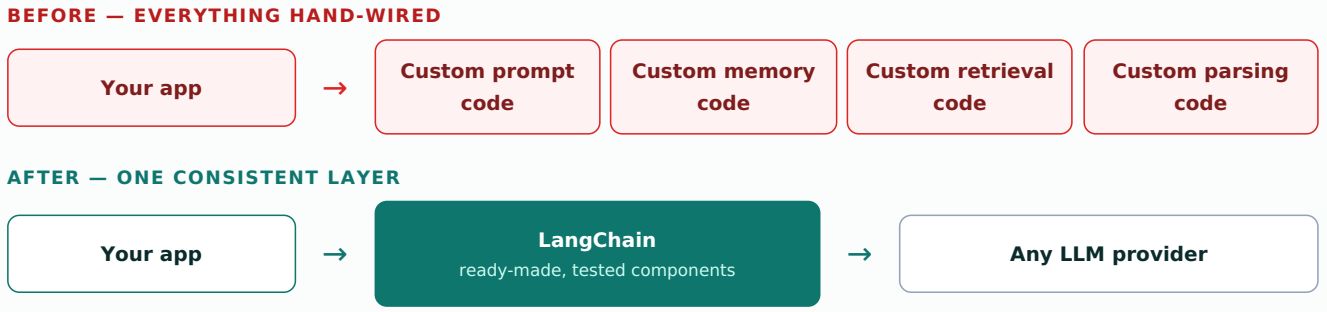

Figure 2 — LangChain replaces hand-written glue code with reusable, standard parts.

3 The Seven Components

Each LangChain component exists to remove one specific pain point. Here is the whole picture before we go through them one at a time.

Component	The pain it removes	What you get instead
LLMs	Every provider has a different interface	One way of calling any model
Prompt Templates	Prompts scattered and duplicated across files	Reusable, versioned templates
Output Parsers	Model returns unpredictable free text	Clean structured data your code can trust
Chains	Multi-step logic written by hand each time	Steps wired together declaratively
Runnables	Components that do not fit together neatly	One shared interface everything speaks
Tools	The model cannot reach the outside world	Access to databases, APIs, your functions
Memory	Every message starts from zero	Conversations that remember context

Component 1 — LLMs

Each provider — OpenAI, Google Gemini, Anthropic — has its own way of being called. Written directly, switching from one to another means rewriting large parts of your application.

LangChain puts a common interface in front of all of them. Your application logic stays the same; only the model choice changes.

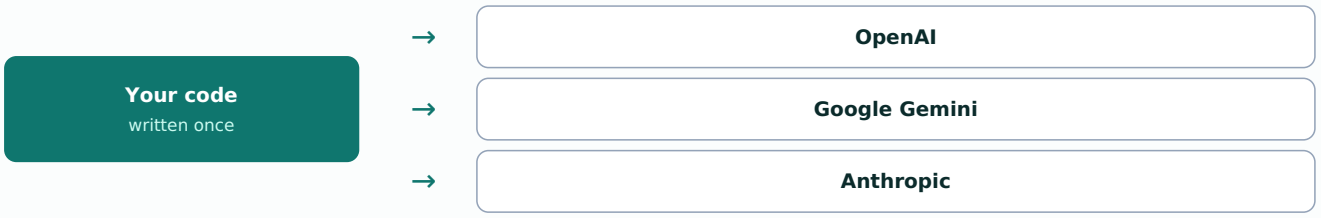

Figure 3 — Swap the provider without rewriting the application.

> **Why this matters in practice:** model pricing, speed and quality change constantly. Being able to test a different provider in minutes rather than days is a real commercial advantage.

Component 2 — Prompt Templates

A prompt is the instruction you send to the model. In a real application the same instruction is reused constantly, with only a few details changing each time.

A **prompt template** is that instruction written once, with blanks in it. You fill the blanks at run time.

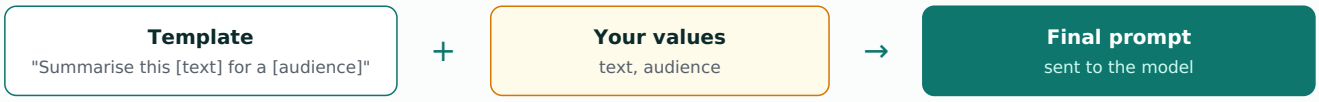

Figure 4 — One template, reused with different inputs every time.

Because the wording now lives in one place, you can improve it once and every part of the application benefits. You can also keep **versions** of a prompt, which matters more than it sounds — a small wording change can noticeably alter output quality, and you want to be able to go back.

Component 3 — Output Parsers

An LLM replies in ordinary text. That is fine for a human reading it, but a problem for your code, which needs predictable fields it can rely on.

An **output parser** sits after the model and converts that reply into structured data — typically JSON, or a validated object defined with Pydantic in Python.

Figure 5 — From text a human reads to data a program can use.

> **The real benefit:** validation. A parser can reject a reply that does not match the shape you asked for, instead of letting a malformed response quietly break something further down the line.

Component 4 — Chains

Chains are what the framework is named after. Most useful applications are not a single model call — they are several steps, where the result of one feeds the next.

A chain lets you describe that flow instead of hand-writing the wiring. There are three common shapes.

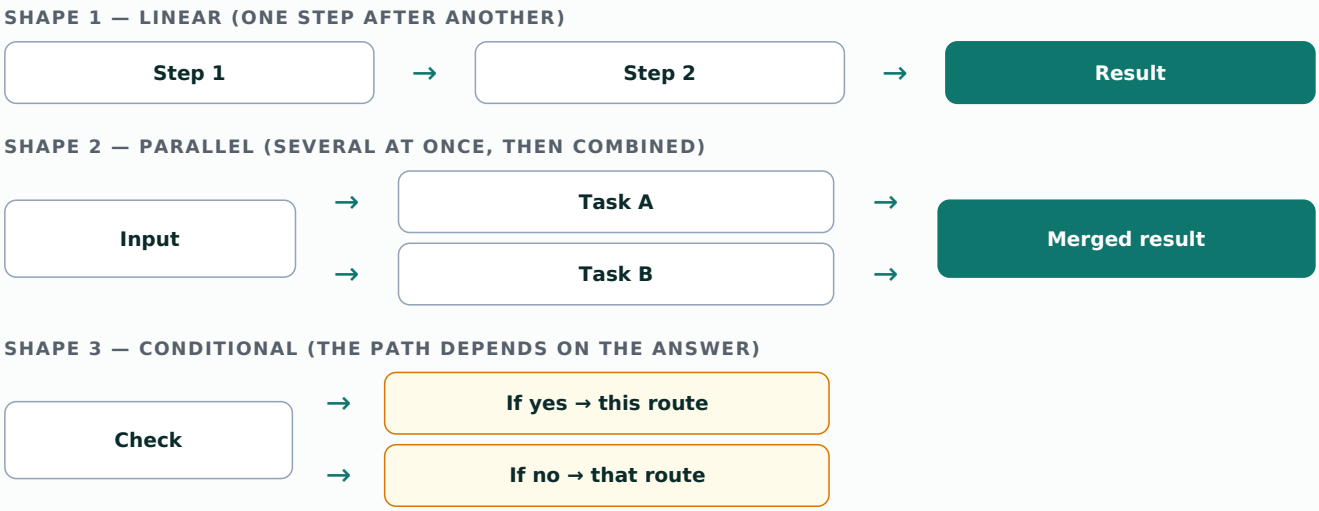

Figure 6 — The three ways steps can be arranged in a chain.

Component 5 — Runnables

Runnables are the least visible component and the most important one structurally.

A **runnable** is simply anything that follows the same standard interface: it takes an input and returns an output. Because prompts, models, parsers and whole chains are all runnables, they can be slotted together in any order without adapter code.

Figure 7 — Same interface everywhere, so pieces connect like building bricks — and a finished chain can itself become one step inside a bigger chain.

Component 6 — Tools

On its own, a language model is sealed off from the world. It cannot look up a live price, query your database, or run one of your own functions.

Tools open that door. You register a function, describe what it does, and the model can then choose to call it when a question requires real information or a real action.

Figure 8 — Tools connect the model to external systems and your own code.

> **Looking ahead:** tools are the foundation that agents are built on. An agent is essentially a model deciding, on its own, which tools to use and in what order.

Component 7 — Memory

LLM calls are **stateless**. Each request is independent, and the model has no idea what was said a moment ago. Without help, a chatbot forgets your name the instant you finish typing it.

LangChain's **memory** components handle chat history for you, passing the relevant earlier context along with each new message.

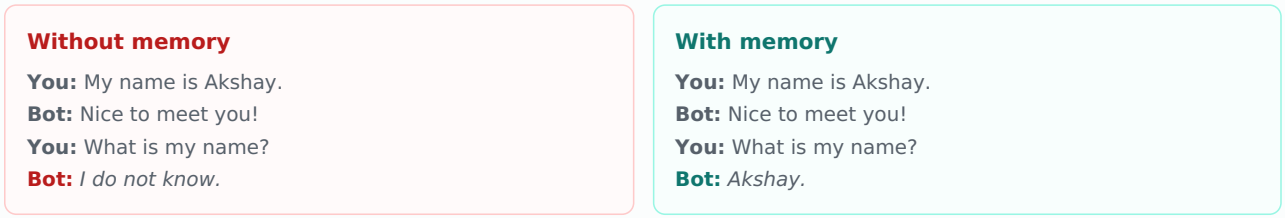

Figure 9 — Memory is what turns isolated calls into an actual conversation.

4 How It All Fits Together

A realistic LangChain application uses most of these components at once.

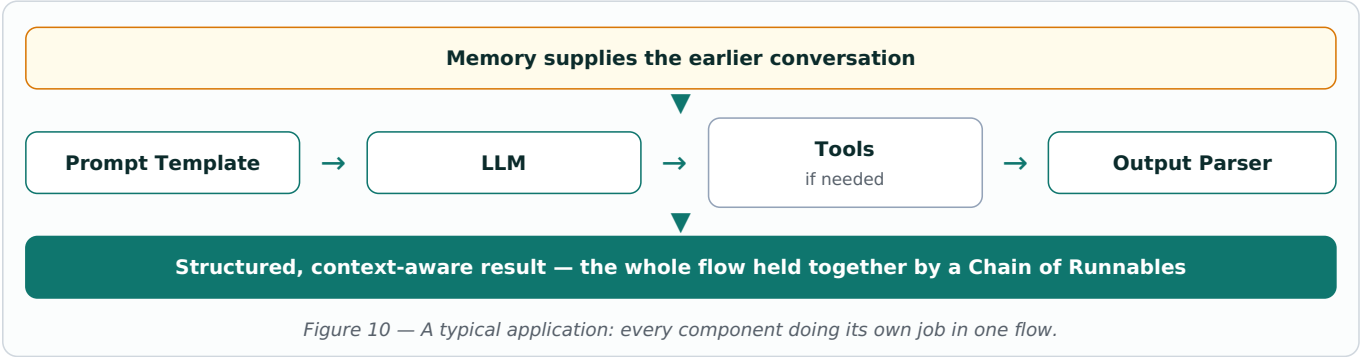

Figure 10 — A typical application: every component doing its own job in one flow.

5 What Comes Next

This video stayed deliberately high level. Nothing here required writing code — the goal was to make sure you know what each piece is *for* before you start using it.

The upcoming lectures move into practice:

- **Hands-on implementation** — actually building with each component covered here.
- **RAG architectures** — connecting the model to your own documents and data.
- **Agentic AI** — systems that plan and act across multiple steps on their own.

> **Before moving on,** make sure you can say in one sentence what each of the seven components does. Everything that follows assumes these names are already familiar.

Quick recall check

If you need to…	Reach for…
Switch from one model provider to another	**LLMs**
Reuse the same instruction with different inputs	**Prompt Templates**
Get reliable JSON instead of loose text	**Output Parsers**
Run several steps in order, in parallel, or conditionally	**Chains**
Make components snap together cleanly	**Runnables**
Let the model query a database or call your function	**Tools**
Keep context across a conversation	**Memory**

> **The short version:** LangChain does not make the model smarter. It removes the repetitive plumbing around the model, so the code you write is the part that is actually specific to your application.